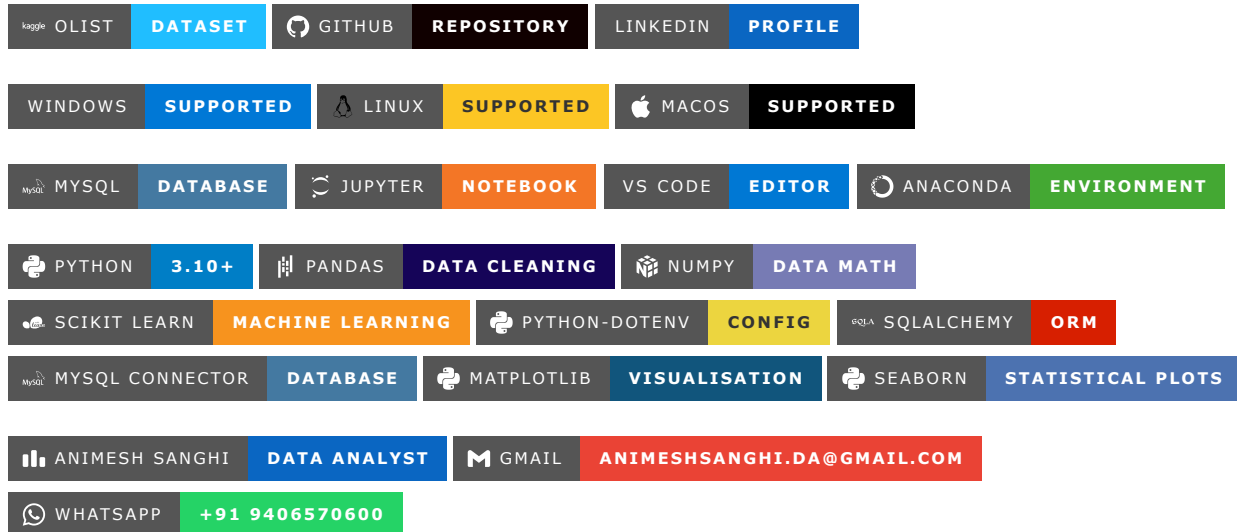


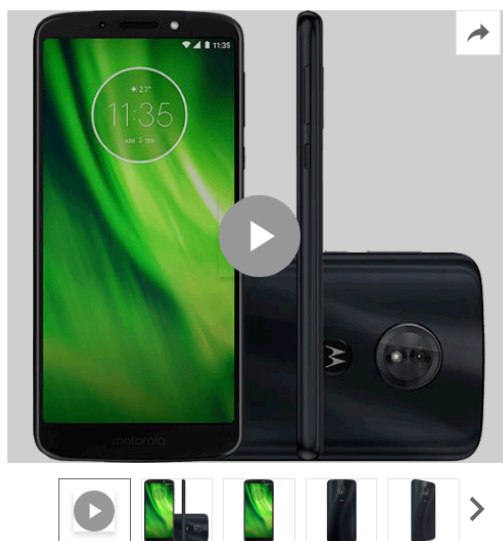
README

Olist E-Commerce Data Analysis

Date of Completion: April 1, 2026



[Olist website](#) view for product listing:



Smartphone Motorola Moto G6 Play Dual Chip Android Oreo - 8.0 Tela 5.7" Octa-Core 1.4 GHz 32GB 4G Câmera 13MP - Índigo

(Cód.133453169) ★★★★★ (215)

☐ Caixa de Som ANKER SoundCore Bluetooth 12W - Preta
+ R\$ 429,99

pegue na loja hoje!

Pegue na loja mais próxima, no mesmo dia :)
Sujeito à alteração de preço. [Saiba mais](#)

[ver lojas](#)

Escolha uma loja abaixo e compre

olist
☒ **R\$ 1.299,00**
R\$ 26,04 - 7 a 10 dias úteis

vendido e entregue por **olist**

R\$ 1.299,00

10x de R\$ 129,90 s/ juros

 **comprar**

Corra! Temos apenas 5 no estoque

on: ra
☐ **R\$ 1.069,90**
R\$ 38,32 - 7 a 10 dias úteis

mel: cê
☐ **R\$ 975,00**
R\$ 22,94 - 5 a 6 dias úteis

Mais opções deste produto a
partir de **R\$ 959,00**

 **R\$ 1.299,00** em até 12x de R\$ 108,25 s/ juros

R\$ 1.299,00 no cartão : em até 24x de R\$ 54,12 s/ juros

[formas de parcelamento](#)

:) Este produto é vendido por uma loja parceira.

Project Overview

This project involves a detailed analysis of a Brazilian E-commerce dataset provided by Olist on Kaggle. The dataset contains multiple tables with over 100k rows of data showing customer behavior, logistics, sales, and product performance.

About the Data: This dataset has over 2 million views and 487K downloads as of April 1, 2026.

It contains information on 100k orders made across multiple marketplaces in Brazil from 2016 to 2018.

This is real, anonymized commercial data. Company and partner names in the reviews have been replaced with Game of Thrones house names.

Note: Column names such as `product_name_lenght` and `product_description_lenght` keep their original spelling from the source database to ensure our data pipeline works correctly.

Acknowledgements:

Thanks to [Olist](#) for releasing this dataset.

Thanks to [Kaggle](#) for making it available.

Approach:

To keep things structured and professional, this project follows the industry-standard 6-stage data analytics framework: **Ask, Prepare, Process, Analyse, Share, and Act.**

Project Report:

You can also explore the entire process without installing any applications by reading the [Olist Project Report.pdf](#).

This report is a complete compilation of all scripts and their outputs, and reviewing it requires no technical setup.

Executive Summary & Business Impact:

👉 View the final presentation: [Markdown Version](#) | [PDF Slides](#)

Key Insights:

- **Late Deliveries Hurt Reviews:** When an order arrives late, customer ratings drop by ~40% (falling from about 4.2 to 2.5 stars). Every extra day of delay lowers the score by another 0.05 stars.
- **Peak Shopping Times:** Most sales happen early in the workweek, specifically Mondays and Tuesdays between 1:00 PM and 4:00 PM. The absolute busiest time is Monday at 4:00 PM, with most people paying by credit card.
- **High Shipping Costs Cause Cancellations:** Expensive delivery fees turn buyers away. If shipping makes up more than 20% of the total order cost, cancellations go up. If it crosses 33%, it hits a "danger zone" where customers are highly likely to cancel their orders.

TL;DR: Key Business Impact

- **Revenue Optimisation:** Proved a 0.32 correlation between payment installments and higher checkout values, guiding new ad spend strategies.
- **Logistics ROI:** Built a Linear Regression model proving that every 1 day of shipping delay mathematically destroys average review scores by ~0.05 stars.
- **Catalog Efficiency:** Disproved the company myth that massive product descriptions improve customer satisfaction.

Prerequisites & Software Requirements

This project was developed on **Windows 10 (64-bit)** using **VS Code**, **Anaconda**, **Jupyter Lab**, and **MySQL**. To run the analysis on your local machine, make sure you have these applications (or their alternatives) installed on your operating system (Windows, Linux, or macOS).

The Tech Stack:

- **Database:** MySQL (for local data storage and extraction)
- **Language:** Python 3.10+
- **Libraries:** Pandas, NumPy, Matplotlib, Seaborn, SQLAlchemy, Scikit-learn, mysql-connector-python, and python-dotenv.

Required Software and Alternatives:

- **MySQL Community Server:** Essential for hosting the local e-commerce database.
(Alternatives: *MariaDB as an open-source replacement, or running MySQL via Docker Desktop*).
- **Anaconda (or Miniconda):** Essential for recreating the isolated Python environment and installing the exact dependencies without system conflicts.
(Alternative: *Standard Python 3.10+ using venv and pip*).

Recommended Tools and Alternatives:

- **MySQL Workbench:** Highly recommended for running the initial SQL scripts and checking the tables visually.
(Alternatives: *DBeaver or JetBrains DataGrip. These are great if MySQL Workbench runs poorly on your Linux or macOS setup*).
- **Jupyter Lab or Jupyter Notebook:** Recommended for opening and running the .ipynb analysis and visualisation files.
(Alternatives: *The VS Code Jupyter Extension, or Google Colab to run notebooks in the cloud*).
- **Visual Studio Code:** A great all-in-one editor for viewing the Python scripts (.py), Markdown files (.md), and SQL queries.
(Alternatives: *PyCharm or Sublime Text*).

Environment Setup (Database Password)

This project connects to a local MySQL database. To avoid typing your password manually every time a script runs, you should set up an environment file:

1. Find the [.env.example](#) file in the main folder.
2. Duplicate it and rename the copy to exactly [.env](#).
3. Open the [.env](#) file and replace `your_mysql_password_here` with your actual MySQL password.

Smart Fallback Feature: If you choose not to use a [.env](#) file, or if the password inside it is wrong, the scripts will not crash. Instead, the system will politely ask you to enter the password manually in the terminal (you get a maximum of 3 attempts).

Environment Setup (Anaconda)

Data science projects need specific versions of libraries (like pandas, scikit-learn, and seaborn). Relying on your computer's default Python can cause version conflicts.

This project uses Anaconda to create a strictly isolated environment. This ensures anyone can run the code smoothly without breaking their local setup.

Project Structure

The repository contains around 50 files. It is organised chronologically to follow the 6 stages of data analytics:

Project & Analyst Overview

- [Olist Project Report.pdf](#) - Comprehensive compilation of all project scripts, methodologies, and outputs. *(Primary Deliverable)*
- [Animesh Resume.pdf](#) - Author's detailed Data Analyst resume.

Documentation & Environment Setup

- [README.md](#) OR [.pdf](#) - Complete project guide, architecture overview, and execution instructions. *(Start Here)*
- [0.1_env_setup_guide.md](#) OR [.pdf](#) - Step-by-step instructions for Python environment and dependency configuration.
- [requirements.txt](#) - Required Python libraries and dependencies for isolated environment setup.
- [.env.example](#) - Template for environment variables; requires local configuration for database credentials.
- [.gitignore](#) - Configuration file for secure GitHub version control.

Stage 1: Ask (Problem Definition)

- [1.0 Stage 1-ASK-Business Questions.md](#) OR [.pdf](#) - Core business problems, stakeholder questions, and defined analytical objectives. *(Core Document)*

Stage 2: Prepare (Database Setup)

- [2.1 Stage 2-PREPARE-Database Setup.sql](#) OR [.pdf](#) - SQL scripts for building the database schema and ingesting raw dataset files. *(Core Script)*

Stage 3: Process (Data Cleaning & Pipeline)

- [utils.py](#) - Custom utility scripts for establishing secure MySQL database connections. *(Core Script)*
- [3.1 Stage 3-PROCESS_Diagnosis_Raw.py](#) - Python script for initial raw data diagnostics and exploration. *(Core Script)*
- [3.2 Stage 3-PROCESS_Diagnosis_Raw_Output.txt](#) - Terminal execution log documenting the raw data diagnosis.
- [3.3 Stage 3-PROCESS_Cleaning_Data.py](#) - Automated Python pipeline for data cleaning, type conversion, and formatting. *(Core Script)*
- [3.4 Stage 3-PROCESS_Cleaning_Data_Output.txt](#) - Terminal execution log for the cleaning pipeline.
- [3.5 Stage 3-PROCESS_Diagnosis_Cleaned.py](#) - Python script to validate and diagnose the cleaned dataset. *(Core Script)*
- [3.6 Stage 3-PROCESS_Diagnosis_Cleaned_output.txt](#) - Terminal execution log for post-cleaning validation.

Stage 4: Analyze (EDA & Modeling)

- [4.0 Stage 4-ANALYSE Analysis.ipynb](#) OR [.pdf](#) - Jupyter Notebook detailing Exploratory Data Analysis (EDA) and predictive machine learning models. (*Primary Deliverable*)

Stage 5: Share (Data Visualization)

- [5.0 Stage 5-SHARE Visualisation Report.ipynb](#) OR [.pdf](#) - Technical notebook documenting the generation of data visualizations and business dashboards. (*Technical Overview*)
- [5.1 Stage 5-SHARE Presentation.md](#) OR [.pdf](#) - Slide deck outlining the visual insights derived from the data. (*Stakeholder Presentation*)

Stage 6: Act (Business Strategy & Conclusion)

- [6.0 Stage 6-ACT Executive Summary Report.md](#) OR [.pdf](#) - Comprehensive written report detailing final business insights and strategic recommendations. (*Technical Overview*)
- [6.1 Stage 6-ACT Presentation.md](#) OR [.pdf](#) - Executive slide deck directly connecting data visualizations to actionable business growth strategies. (*Stakeholder Presentation*)

🚀 How to Run the Project (Start to Finish)

Follow these steps to run the project locally from scratch:

Note: Please don't confuse "steps" and "stages" as they mean different things in this project.

[Download the dataset from Kaggle:](#)

The screenshot shows the Kaggle dataset page for 'Brazilian E-Commerce Public Dataset by Olist'. The page has a dark theme. At the top, there's a search bar and navigation links. The dataset title is prominently displayed, along with a description: '100,000 Orders with product, customer and reviews info'. Below the title, there are tabs for 'Data Card', 'Code (718)', 'Discussion (69)', and 'Suggestions (0)'. The 'About Dataset' section is visible, containing a welcome message, details about the dataset's origin (Olist Store), and information about its features. A 'Usability' score is shown as a modal overlay, indicating 100% scores for Completeness, Credibility, and Compatibility, with a list of specific attributes checked. The 'Context' section at the bottom explains that the data was provided by Olist, a large department store in Brazil.

Step 1: Clone the Repository

1. **Clone the Repository:** Download or clone this project from GitHub to your local machine.
2. **Prepare the Data:** Inside the main project folder, create a new folder named data. Extract and save all the CSV files downloaded from Kaggle into this data folder.
3. **Launch Terminal:** Open the main project folder in your terminal or command prompt.
Example (*replace the path with your actual folder path*):

```
cd C:\Users\MY PC\Downloads\Olist_project_by_animesh
```

Step 2: Setup the Python Environment

This project uses an Anaconda environment to prevent library conflicts. In your terminal, run these commands one by one:

```
conda create --name animesh_env --file requirements.txt
```

```
conda activate animesh_env
```

```
python -m ipykernel install --user --name=animesh_env --display-name="Python (E-commerce Project)"
```

Check the [0.1_env_setup_guide.md](#) or [.pdf](#) file for more details.

Step 3: Configure Database Credentials

Set up your [.env](#) file as instructed in the Environment Setup section above so the Python scripts can safely access your local database.

Step 4: Setup the Database (MySQL)

Open MySQL Workbench (or your preferred SQL client).

Open and run the [2.1_Stage_2-PREPARE-Database_Setup.sql](#) file.

This creates the `ecomm_analytics` database, builds the tables, and loads the raw CSV files into your local server.

Step 5: Run the Data Pipeline (Processing & Cleaning)

Activate `animesh_env` and run these Python scripts in your terminal (or an editor like VSCode) or preferred app to [diagnose](#), [clean](#), and [prepare](#) the data:

1. `python 3.1_Stage_3-PROCESS_Diagnosis_Raw.py`
2. `python 3.3_Stage_3-PROCESS_Cleaning_Data.py`
3. `python 3.5_Stage_3-PROCESS_Diagnosis_Cleaned.py`

Step 6: Explore the Analysis & Visualisations

Open Jupyter Lab (or your preferred app) and make sure your kernel is set to `Python (E-commerce Project)`. Then, open and run all cells in these notebooks:

1. [4.0_Stage_4-ANALYSE_Analysis.ipynb](#) – to view the EDA and predictive model.
2. [5.0_Stage_5-SHARE_Visualisation_Report.ipynb](#) – to generate business charts (this also saves them as PNGs).

Just want to see the results? Check out the PDFs: [analyse](#) and [share](#).

Step 7: Explore the Presentations and Act Report

Open VS Code (or your preferred app) to review the final presentations and reports:

1. [5.1_Stage_5-SHARE_Presentation.md](#): A presentation showing the key visual insights for stakeholders.
2. [6.0_Stage_6-ACT_Executive_Summary_Report.md](#): A written report on the best steps for business growth.
3. [6.1_Stage_6-ACT_Presentation.md](#): A presentation connecting the data visuals directly to business strategies.

Just want a quick look? Check out the PDFs: [stage-5 presentation](#), [stage-6 report](#), and [stage-6 presentation](#).

Stage 1: Ask (Defining the Business Problem)

The Business Objective: To find actionable insights regarding sales trends, shipping delays, customer buying behavior, and catalog optimisation to drive revenue, improve delivery times, and keep customers coming back.

Key Analytical Focus Areas:

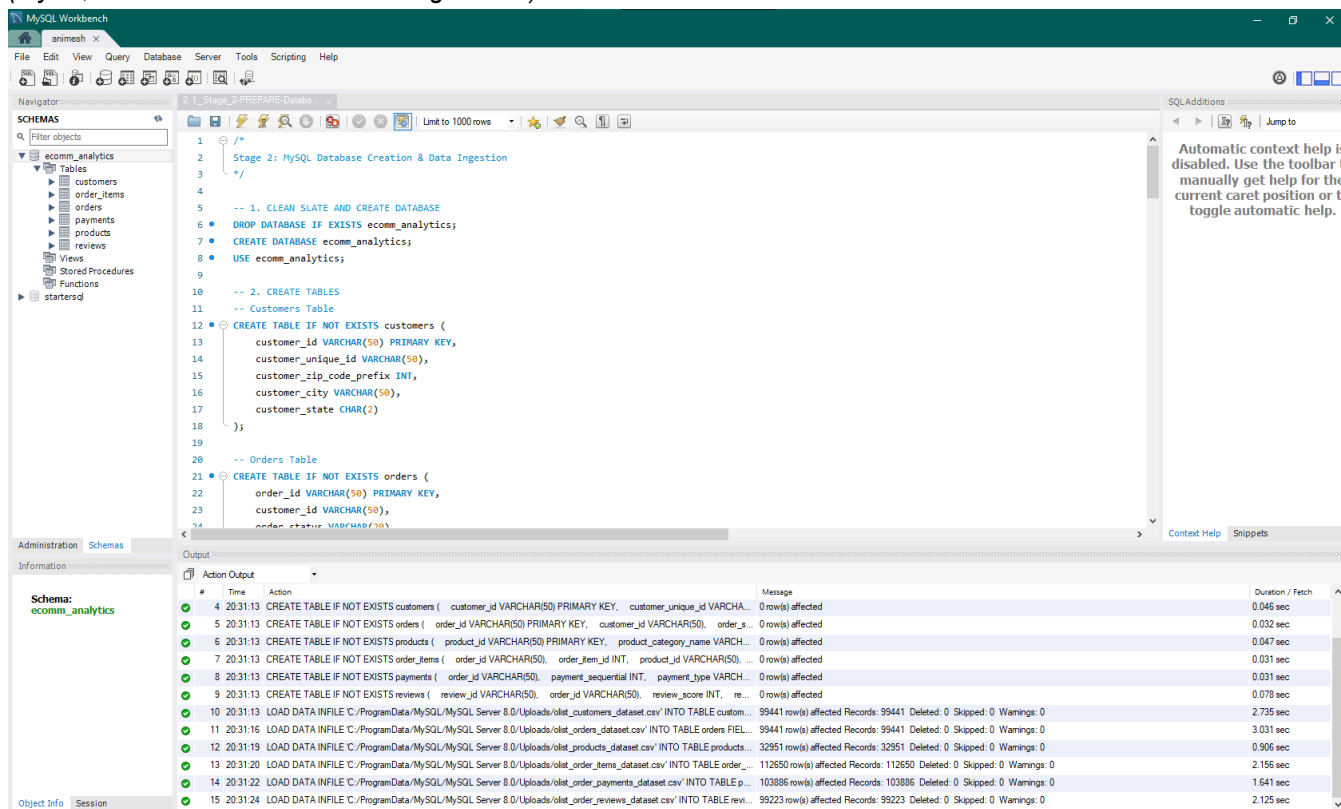
1. **Sales & Revenue Trends:** Analysing Month-over-Month (MoM) growth, peak traffic times (day/hour), and Average Order Value (AOV).
2. **Logistics & Operational Efficiency:** Checking average delivery times and finding out which regions are the slowest.
3. **Customer Behavior & Segmentation:** Understanding how people pay and checking if payment installments lead to bigger orders.
4. **Product Catalog & Satisfaction:** Comparing sales volume vs. revenue across categories and testing if longer descriptions or more photos actually improve review scores.
5. **Predictive Impact Analysis:** Using Linear Regression to mathematically calculate the "Late Penalty"—meaning exactly how much a review score drops for every extra day a delivery is late.

Note: All core business questions from stakeholders and the specific analytical questions addressed in this project are listed in the [1.0 Stage 1-ASK-Business_Questions.md](#) or [.pdf](#) file.

🔧 Stage 2: Prepare (Setting Up & Gathering Data)

The raw data comes from the [Brazilian E-Commerce Public Dataset by Olist](#). Raw CSV files were successfully loaded into a **MySQL database** for safe storage and easy querying.

(MySQL Workbench view after loading all files)



Core Tables Used:

	Table Name	Description	Key Columns
1.	Orders	Core transaction data.	order_id, customer_id, timestamp
2.	Order_Items	Specific items per order.	order_id, product_id, price
3.	Products	Product catalog details.	product_id, category_name
4.	Customers	Buyer demographics.	customer_id, zip_code, city
5.	Reviews	Customer feedback.	review_score, review_comment
6.	Payments	Payment methods used.	payment_type, payment_value

Database Setup:

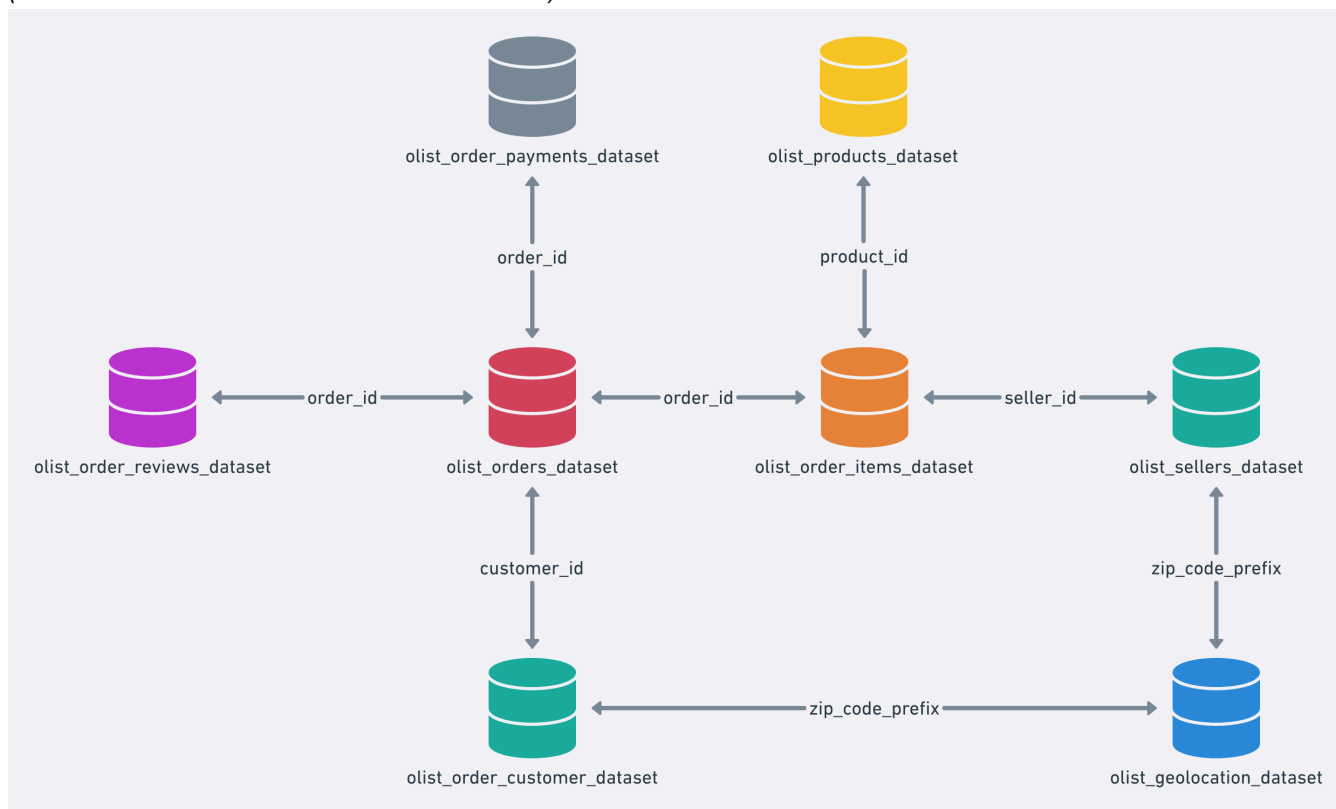
1. Created a centralized relational database in [MySQL Workbench](#).
2. Set up the right schema and tables for the raw files.
3. Loaded the data using the `INFILE` command.

Note: The SQL queries for creating the database and loading data are in the [2.1_Stage_2-PREPARE-Database_Setup.sql](#) file.

🔧 Stage 3: Process (Cleaning the Data)

Data cleaning was done using [Python](#) ([Pandas](#) & [NumPy](#)) connected straight to the MySQL database. Python was chosen over pure [SQL](#) for this step because its libraries make complex changes and outlier handling much easier.

(Schema of column names in the dataset tables)



Security First: Secure MySQL Password Handling in Python:

Database Security: [utils.py](#), [getpass](#), & [load_dotenv](#)

- **Why it was created:** Hardcoding database passwords is a major security risk, especially when sharing code in a [public GitHub repo](#).
- **The Setup:** I created a standalone script ([utils.py](#)) using the [sqlalchemy](#), [getpass](#), [logging](#), [load_dotenv](#), and [os](#) libraries.

How it Works:

The script securely handles your MySQL password using a two-step logic:

1. **Automated:** It first tries to read the password from a [.env](#) file (following the [Environment Setup steps](#)). In this process, [load_dotenv](#) loads the file, [os](#) checks for its existence to extract the password, and then [sqlalchemy](#) uses that password to establish the connection.
2. **Manual Fallback:** If the [.env](#) file is missing or has the wrong password, the [getpass](#) library creates a hidden prompt in the terminal for manual entry.

Attempt Logic:

You have a total of **3 attempts** to enter the correct password:

- If a [.env](#) file exists but contains the wrong password, it counts as an attempt.
- If the [.env](#) file does not exist, it does not count as an attempt, and you proceed to manual entry.

How to Use it:

Instead of typing passwords into analysis files, simply import the connection function at the top of your notebooks:

```
from utils import get_db_connection
engine = get_db_connection()
```

Staying Safe on GitHub:

To keep personal credentials off the web, the password stays in the .env file. I created a .gitignore file to ensure this file is automatically excluded during a git push.

Note: .gitignore only works for command-line pushes; if you upload files manually to GitHub, ensure you do not include the .env file.

The 6-Step Cleaning Workflow:

1. **Data Loading & Initial Cleaning:** Pulled all 6 tables from MySQL and formatted them for processing.
2. **Correcting Data Types & Optimisation:**
 - Fixed dates and times to ensure time-series analysis works correctly.
 - Converted specific numeric columns using `.astype('float32')`.
 - **Why?** This significantly reduces memory usage. Switching from the default 64-bit to 32-bit keeps the notebook lightweight and improves processing speed, which is essential when handling large datasets.
3. **Feature Engineering:** Created new, helpful columns to enable deeper business analysis.
4. **Duplicate & Outlier Removal:** Cleaned up repeating data and extreme values to keep the statistical math accurate.
5. **Data Aggregation & Merging:** Combined all individual clean tables into one central `master_analytical_view`.
6. **Exporting to MySQL:** Sent the cleaned tables and the new master view back to the database. This keeps the original raw data safe while providing a "single source of truth" for the analysis phase.

Use these scripts depending on what you want to check:

Task	Script File & Expected Output
Diagnosis of Raw Data	3.1 Stage 3-PROCESS_Diagnosis_Raw.py & Expected Output
Cleaning	3.3 Stage 3-PROCESS_Cleaning_Data.py & Expected Output
Diagnosis of Cleaned Data	3.5 Stage 3-PROCESS_Diagnosis_Cleaned.py & Expected Output

(VS Code view while cleaning the data)

The image shows a VS Code editor with a Python script named `3.3_Stage_3-PROCESS_Cleaning_Data.py` open. The script is divided into two main sections: `1. DATA LOADING AND INITIAL CLEANING` and `2. CORRECTING DATA TYPES`. The first section involves connecting to a MySQL database, fetching data from several tables, and loading them into pandas DataFrames. The second section involves cleaning the data, such as removing duplicates and correcting data types.

```
1 ...
2 Stage 3.2: Data Cleaning & Feature Engineering Pipeline
3 ...
4
5 import pandas as pd
6 import numpy as np
7 from utils import get_db_connection
8
9 # =====
10 # 0. DATABASE CONNECTION SETTINGS
11 # =====
12 print("**40, \"(no. DATABASE CONNECTION SETTINGS)\", \"**40)
13
14 # Call the function from your new utils.py file
15 engine = get_db_connection()
16 if not engine:
17     print("Exiting script due to connection failure.")
18     exit()
19 print("Process 1/4 Complete: Database connection engine created.\n")
20
21 # =====
22 # 1. DATA LOADING AND INITIAL CLEANING
23 # =====
24 print("**40, \"(no. DATA LOADING AND INITIAL CLEANING)\", \"**40)
25
26 # =====
27 # 1.1. DATA LOADING
28 # =====
29 # List of all tables we need to extract from the MySQL database
30 tables = ["customers", "orders", "products", "order_items", "payments", "reviews"]
31 # Initialize an empty dictionary to hold our dataframes
32 dfs = {}
33
34 print("Process started: Loading tables from MySQL database...")
35 # Loop through the table list and query all data for each one
36 for table in tables:
37     print(f" -> Fetching table: {table}...")
38     # SQL query to select everything from the current table
39     query = f"SELECT * FROM {table}"
40     # Read the SQL query results directly into a pandas DataFrame and store it in the dictionary
41     dfs[table] = pd.read_sql(query, engine)
42     print(f" -> Successfully loaded {table} ({len(dfs[table])} rows).")
43 print("Process complete: All tables loaded successfully.\n")
44
45 print("Process started: Extracting all dataframes for cleaning...")
46 # Extract each dataframe from the dictionary into its own dedicated variable.
47 # Using .copy() ensures these are independent objects, preventing 'SettingWithCopy'
48 # warnings later.
49 orders = dfs['orders'].copy()
50 products = dfs['products'].copy()
51 customers = dfs['customers'].copy()
52 order_items = dfs['order_items'].copy()
53 payments = dfs['payments'].copy()
54 reviews = dfs['reviews'].copy()
55 print("Process complete: All dataframes extracted.\n")
56
57 # =====
58 # 1.2. CLEANING THE ORDERS TABLE
59 # =====
```

The terminal output shows the execution of the script, including the database connection settings, the data loading process, and the cleaning steps. The output indicates that the database connection was successful, and the data was loaded from the MySQL database. The cleaning process is also shown, with the script successfully loading all tables and extracting the dataframes.

Chat: ython.exe" d:/01_project_e-com/3.3_Stage_3-PROCESS_Cleaning_Data.py

Author: Animesh Sanghi

Title: Data Analyst | MBA Candidate in Analytics & Data Science
Phone: 94057000
Email: sanghani789@gmail.com
LinkedIn: <https://www.linkedin.com/in/animesh-sanghi/>
GitHub: <https://github.com/sanghani789>

(animesh_env) PS D:\01_project_e-com & "C:/Users/My PC/anaconda3/envs/animesh_env/pyt
hum.exe" d:/01_project_e-com/3.3_Stage_3-PROCESS_Cleaning_Data.py

0. DATABASE CONNECTION SETTINGS
=====

INFO: Successfully connected to the MySQL database.
Process 1/4 complete: Database connection engine created.

=====

1. DATA LOADING AND INITIAL CLEANING
=====

Process started: Loading tables from MySQL database...
-> Fetching table: customers...
-> Successfully loaded customers (99441 rows).
-> Fetching table: orders...
-> Successfully loaded orders (99441 rows).
-> Fetching table: products...
-> Successfully loaded products (32951 rows).
-> Fetching table: order_items...
-> Successfully loaded order_items (112650 rows).
-> Fetching table: payments...
-> Successfully loaded payments (103886 rows).
-> Fetching table: reviews...
-> Successfully loaded reviews (99223 rows).
Process complete: All tables loaded successfully.

Process started: Extracting all dataframes for cleaning...
Process complete: All dataframes extracted.

Process started: Cleaning missing delivery and approval dates in 'orders'...
Process complete: Orders cleaned. New count: 96476.

Process started: Cleaning missing metrics and dimensions in 'products'...
Process complete: Products cleaned. New count: 32949.

Process started: Standardizing empty strings in 'reviews'...
Process complete: Empty strings converted to NaN in reviews.

=====

2. CORRECTING DATA TYPES
=====

Process started: Converting date and categorical columns across all tables...
Process complete: All data types standardized.

Ln 1, Col 1 Spaces: 4 UTF-8 CR LF Python 3.14.3 (animesh_env) Python 3.13 (64-bit)

Stage 4: Analyse (Finding the Patterns)

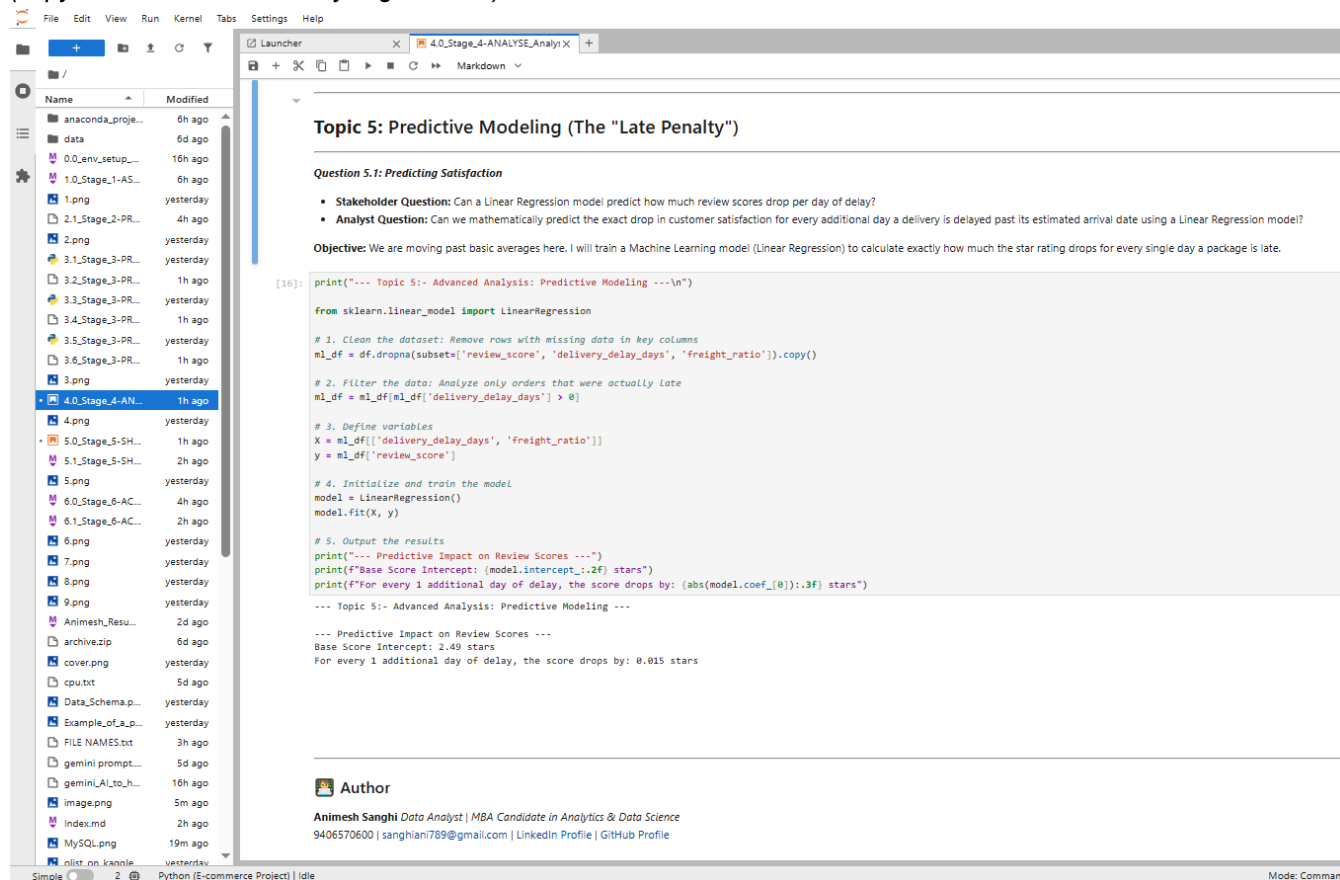
In this step, the cleaned `master_analytical_view` was loaded into Jupyter Lab. Using Pandas, the data was thoroughly analysed to answer the business questions set in Stage 1.

Exploratory Data Analysis (EDA) was done to find links between payment installments and cart sizes, calculate the exact percentage of late deliveries, and find the best-selling product categories.

Predictive Modeling: [Scikit-learn](#) was used to build a Linear Regression model. This helped mathematically figure out the "Late Penalty" to predict exactly how much review scores drop when deliveries are late.

The full analysis and model training can be found in the [4.0_Stage_4-ANALYSE_Analysis.ipynb](#) file.

(Jupyter Lab view while Analysing the data)



The screenshot shows the Jupyter Lab interface. On the left is a file explorer with a list of files and folders, including 'anaconda_proje...', 'data', '0.0_env_setup...', '1.0_Stage_1-AS...', '1.png', '2.1_Stage_2-PR...', '2.png', '3.1_Stage_3-PR...', '3.2_Stage_3-PR...', '3.3_Stage_3-PR...', '3.4_Stage_3-PR...', '3.5_Stage_3-PR...', '3.6_Stage_3-PR...', '3.png', '4.0_Stage_4-AN...', '4.png', '5.0_Stage_5-SH...', '5.1_Stage_5-SH...', '5.png', '6.0_Stage_6-AC...', '6.1_Stage_6-AC...', '6.png', '7.png', '8.png', '9.png', 'Animesh_Resu...', 'archive.zip', 'cover.png', 'cpu.txt', 'Data_Schema.p...', 'Example_of_a_p...', 'FILE NAMES.txt', 'gemini prompt...', 'gemini_AI_to_h...', 'image.png', 'index.md', 'MySQL.png', and 'nlist_nn_kanola...'. The central notebook editor displays a markdown document titled 'Topic 5: Predictive Modeling (The "Late Penalty")'. The document content includes a question about predicting satisfaction, a stakeholder question, an analyst question, an objective, and a code block for a linear regression model. The code block is as follows:

```
[16]: print("--- Topic 5:- Advanced Analysis: Predictive Modeling ---\n")

from sklearn.linear_model import LinearRegression

# 1. Clean the dataset: Remove rows with missing data in key columns
m1_df = df.dropna(subset=['review_score', 'delivery_delay_days', 'freight_ratio']).copy()

# 2. Filter the data: Analyze only orders that were actually late
m1_df = m1_df[m1_df['delivery_delay_days'] > 0]

# 3. Define variables
X = m1_df[['delivery_delay_days', 'freight_ratio']]
y = m1_df['review_score']

# 4. Initialize and train the model
model = LinearRegression()
model.fit(X, y)

# 5. Output the results
print("--- Predictive Impact on Review Scores ---")
print(f"Base Score Intercept: {model.intercept_[0]:.2f} stars")
print(f"For every 1 additional day of delay, the score drops by: {abs(model.coef_[0]):.3f} stars")

--- Topic 5:- Advanced Analysis: Predictive Modeling ---

--- Predictive Impact on Review Scores ---
Base Score Intercept: 2.49 stars
For every 1 additional day of delay, the score drops by: 0.015 stars
```

Below the code block, the document includes an 'Author' section with the name 'Animesh Sanghi' and contact information: '9406570600 | sanghani789@gmail.com | LinkedIn Profile | GitHub Profile'.

Stage 5: Share (Visualising the Insights)

Numbers alone are hard to read. To turn backend data into front-end business value, the main findings were graphed using Matplotlib and Seaborn.

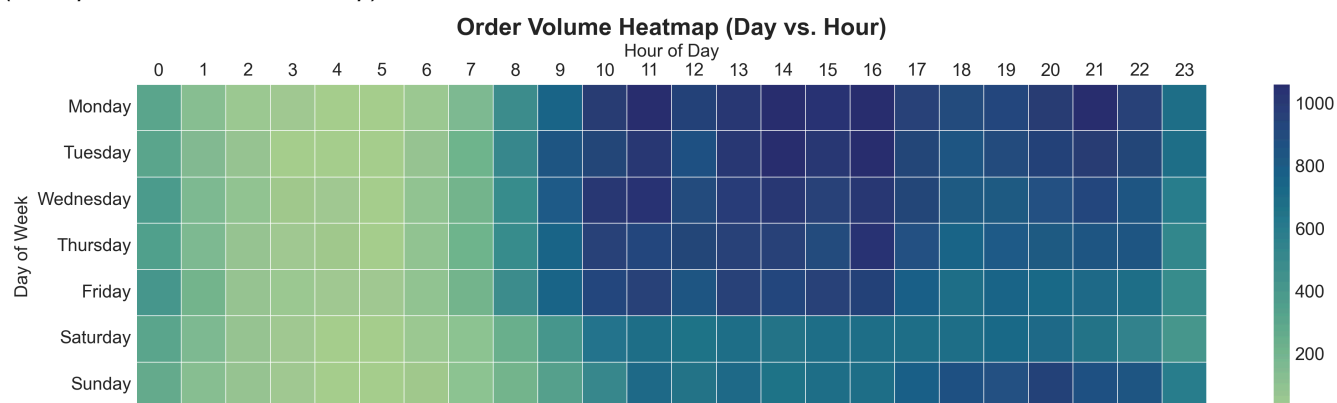
This stage includes 10+ clear, easy-to-read charts that show traffic peaks, delivery slowdowns, category earnings, and just how badly shipping delays hurt customer satisfaction.

The code used to generate the charts and the visualisations themselves can be found in the [5.0 Stage 5-SHARE_Visualisation_Report.ipynb](#) file.

I have also prepared a [5.1 Stage 5-SHARE_Presentation.md](#) file designed specifically for stakeholder meetings.

Both the [report](#) and [presentation](#) are available in PDF format.

(Example: Peak Traffic Heatmap)



💡 Stage 6: Act (Business Recommendations)

Based on the data, here are the strategic recommendations to help the business grow and fix operational issues:

1. **Strategic Timing & Payments:** Move ad spending to weekday afternoons (when traffic is highest) and strongly promote payment installment plans, which are proven to lead to bigger orders.
2. **Fix the "Late Penalty":** Do a full check on the shipping partners handling the Northern/Northeastern states. Our math shows that every extra day of delay costs the business ~0.05 stars, dropping average reviews from 4.29 to 2.27 and ruining trust.
3. **Optimise the Catalog:** Focus on keeping fast-selling items (like Health/Beauty and Bed/Bath) in stock. Stop spending time writing super long product descriptions, as the data shows they don't actually improve review scores.
4. **Launch a VIP Program:** Create a loyalty program with faster shipping for the top 5% of spenders who live in Rio de Janeiro, São Paulo, and Vitória.
5. **Implement a Retention Strategy:** Shift focus from just getting new customers to keeping the ones we have. Over 90% of buyers only ordered once; we need automated post-purchase discounts to get them to buy again.
6. **Deploy AI Operations:** Build an AI system using smart chatbots to handle basic customer support and active order tracking. This will cut down on staff costs and solve customer problems before they leave a 1-star review.

| Read the full business strategy in the [6.0 Stage 6-ACT Executive Summary Report.md](#) file.

| I have also prepared a [6.1 Stage 6-ACT Presentation.md](#) file designed specifically for stakeholder meetings.

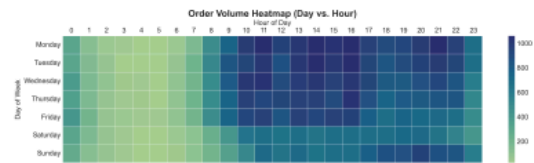
| Both the [report](#) and [presentation](#) are available in PDF format.

Stage 6: Strategic Action Plan & Executive Presentation

Prepared By: Animesh Sanghi

Objective: To translate our data insights into direct, revenue-generating business actions.

📊 Strategic Action 1: Drive Revenue Through Strategic Timing & Payments

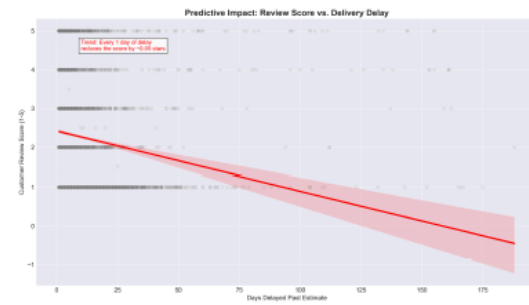


The Insight: Our average order value (AOV) is steady, but our store gets the most traffic and sales on weekday afternoons (specifically Tuesdays around 16:00). Also, the data shows a clear positive link (0.32 correlation) between offering payment installments and customers placing larger, more expensive orders.

The Action: * **Targeted Ads:** Move the biggest part of the digital marketing budget to run during weekday afternoons when people are already buying.

- **Push Installments:** Highlight "0% EMI" and credit card installment options clearly on the checkout page to encourage customers to spend more.

🚚 Strategic Action 2: Fix the "Late Penalty" to Protect Customer Trust

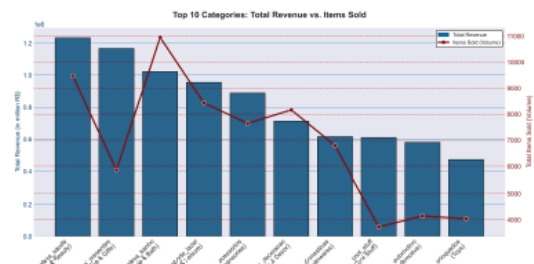


The Insight: Across the whole system, 6.77% of deliveries are late. When a package arrives late, the average customer review drops terribly from 4.29 stars down to 2.27 stars. Using a Machine Learning model (Linear Regression), I found that for every single day a package is delayed, the product's rating mathematically drops by ~0.05 stars.

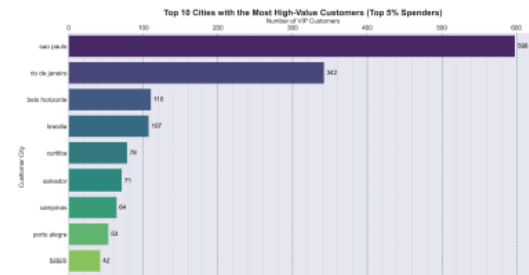
The Action: * **Investigate Shippers:** Start an immediate review of our shipping partners in the Northern and Northeastern states (like RR, AP, and AM), because deliveries there are taking up to 28 days.

- **Adjust Estimates:** Automatically add 3 extra days to the "Estimated Delivery Date" shown to customers in these slow regions. The data proves that delivering early on a longer promise keeps customers happier than missing a short deadline.

📦 Strategic Action 3: Optimize the Catalog and Ignore Myths



📍 Strategic Action 4: Launch a Geo-Targeted VIP Program



Conclusion

This project was a great chance to use the full 6-stage data analytics framework on a massive, real-world e-commerce dataset. By taking the data all the way from raw CSV files to presenting real business strategies, I was able to show how data isn't just about math—it's about making changes that help the business. The insights found here show how looking closely at data can optimise shipping, marketing, and customer happiness.

Acknowledgments

Thank you for taking the time to review my project!

A special thanks to **Olist** and the **Kaggle community** for sharing this great dataset. It is an amazing resource for practicing with real-world data problems.

If you have any feedback, suggestions, or just want to chat about data, analytics, and business strategy, please feel free to reach out!

Author

[Animesh Sanghi](#) | *Data Analyst* | *MBA '28 (Analytics & Data Science) MUJ*
9406570600 | animeshsanghi.da@gmail.com | [LinkedIn Profile](#) | [GitHub Profile](#)